

GPU 推理效率综述：从 SM/MFU 原理到 NVIDIA 卡型上的提升之道

GPU 推理效率综述：从 SM/MFU 原理到 NVIDIA 卡型上的提升之道

面向 LLM / 推荐模型推理工程师。以 NVIDIA GPU (Ampere → Hopper → Blackwell) 为主轴，讲清楚一个反直觉的现实：推理时 GPU "看着满载"，算力却大量空转，以及业界把利用率抬上去的全套办法。

调研日期：2026-06-15。硬件规格与前沿方案均标注来源；自绘示意图为 SVG，引用第三方数据见文末参考文献。SVG/外部数据公开发表前请按各来源 license 复核。

引言：一个 99% 利用率的谎言

运维盯着监控大盘，nvidia-smi 上 GPU-Util 写着 99%，于是汇报"GPU 已经打满，要扩容"。但同一时刻，这张卡真正用在矩阵乘法上的算力可能只有理论峰值的 15%。剩下的 84% 去哪了？等数据。

这不是个别现象，而是推理负载的常态。问题出在"利用率"这个词被严重透支了——它在不同工具、不同语境里指代完全不同的东西，而最容易被看到的那个 (nvidia-smi 的 GPU-Util) 恰恰是信息量最低的。一篇被反复引用的文章把这种现象直接命名为"你的监控面板在对你撒谎" (packet.ai: GPU Utilization: The Lie Your Dashboard Tells You) [G3]。

这篇综述就围绕两个被混为一谈的指标展开：**SM 占用 / 利用** 和 **MFU**。前者描述 GPU 的流式多处理器忙不忙，后者描述算力有没有真正兑现成有用计算。理解它们为什么背离、以及怎么把它们一起抬上去，是推理性能工程的核心。

全文分三部分：

- **第一部分 (§1–§4) 原理地基**：GPU 体系结构、SM、四层利用率、Roofline——讲清楚"是什么"和"为什么推理 MFU 低"。
- **第二部分 (§5–§8) 常见提升办法**：批处理、FlashAttention、算子融合、低精度、并行——讲清楚"怎么做"。
- **第三部分 (§9–§12) 前沿与实战**：分离式推理、NVIDIA 卡型横评、Profiling 实操、趋势展望——讲清楚"业界走到哪"。

第一部分：原理地基

1. 先把"利用率"这个词拆开

日常说"GPU 利用率高不高"，其实问的可能是四个不同层次的东西。它们构成一个从宽松到严格的漏斗，每一层都比上一层更接近"算力到底有没有被用上"的真相。[互动模拟器](#)

四层"利用率"漏斗：越往下越严格

每一层之间的落差，就是一类可优化的损耗

① GPU-Util (nvidia-smi)

采样窗口内"有没有 kernel 在跑" · 典型读数 95~100% · 几乎无诊断价值

~99%

↓ 损耗：kernel 间隙 / 启动开销 / 通信等待

② SM Active (DCGM sm_active)

SM 上有 warp 活跃的时间占比 · 典型 60~85%

~70%

↓ 损耗：访存等待 / softmax、elementwise 等非 GEMM 算子

③ Tensor Active (pipe_tensor_active)

Tensor 管线真正在算的周期占比 · 典型 40~50%

~45%

↓ 损耗：GEMM 形状不优 (小 batch / 奇异维度打不满 tile)

④ MFU

有效 FLOPs / 理论峰值

~30%

示意值

图 1：四层"利用率"漏斗（自绘）。从上到下越来越严格，层与层之间的落差对应一类可优化的损耗。数值为示意。

第一层：GPU-Util (nvidia-smi)。它的定义朴素到容易误解——过去一个采样窗口里，有没有至少一个 kernel 在 GPU 上执行。只要有一个 kernel 在跑，哪怕它只用了 1 个 SM、只做了一次加法，这个数也会接近 100%。所以推理服务一旦在持续出 token，这个数几乎恒定在 95% 以上，对"算力用得好不好"基本没有诊断价值。NVIDIA 自己的 DCGM 文档和多篇分析都强调过这一点 [G1][G3][G5]。

第二层：SM Active (DCGM sm_active 或 DCGM_FI_PROF_SM_ACTIVE)。它衡量的是：在所有 SM 里，平均有多大比例的 SM 在该时间段内至少有一个 warp 驻留。这比第一层严格——如果你的 kernel 只用了 132 个 SM 里的 10 个，这个数会反映出来。但它仍然只问"SM 上有没有活儿"，不问"这个活儿是不是高效的张量计算"。

第三层：Tensor Active（`sm_pipe_tensor_op_active` 一类）。Tensor Core 是现代 GPU 算力的绝对主力（一张 H100 的 FP16 张量算力是它 CUDA Core FP32 算力的十几倍）。这一层问的是：Tensor 管线真正在做矩阵乘累加的周期占比。一个被访存卡住、或者在跑 softmax/LayerNorm 这类非矩阵运算的 kernel，SM Active 可以很高，但 Tensor Active 很低。

第四层：MFU (Model FLOPs Utilization)。最严格，也最诚实。它直接用"模型实际执行的有用浮点运算量"除以"GPU 理论峰值浮点运算量"。下一节专门讲它的算法。

这四层的关系是单调递减的：

$$\text{GPU-Util} \geq \text{SM Active} \geq \text{Tensor Active} \geq \text{MFU}$$

关键不在于每个数本身，而在于相邻两层的落差。每一道落差都对应一类性能损耗，也对应一类优化手段：

落差	损耗来源	对应优化方向
GPU-Util → SM Active	kernel 启动间隙、host-device 同步、通信等待、部分 SM 闲置	CUDA Graphs、算子融合、并行度调整
SM Active → Tensor Active	访存等待、softmax/elementwise 等非 GEMM 算子占时间	FlashAttention、kernel 融合、提高算术强度
Tensor Active → MFU	GEMM 形状不优（小 batch / 维度打不满 tile）、精度选择	增大 batch、对齐张量维度、低精度

记住这张表，后面第二、三部分的每一个技术，本质上都是在缩小其中某一道落差。

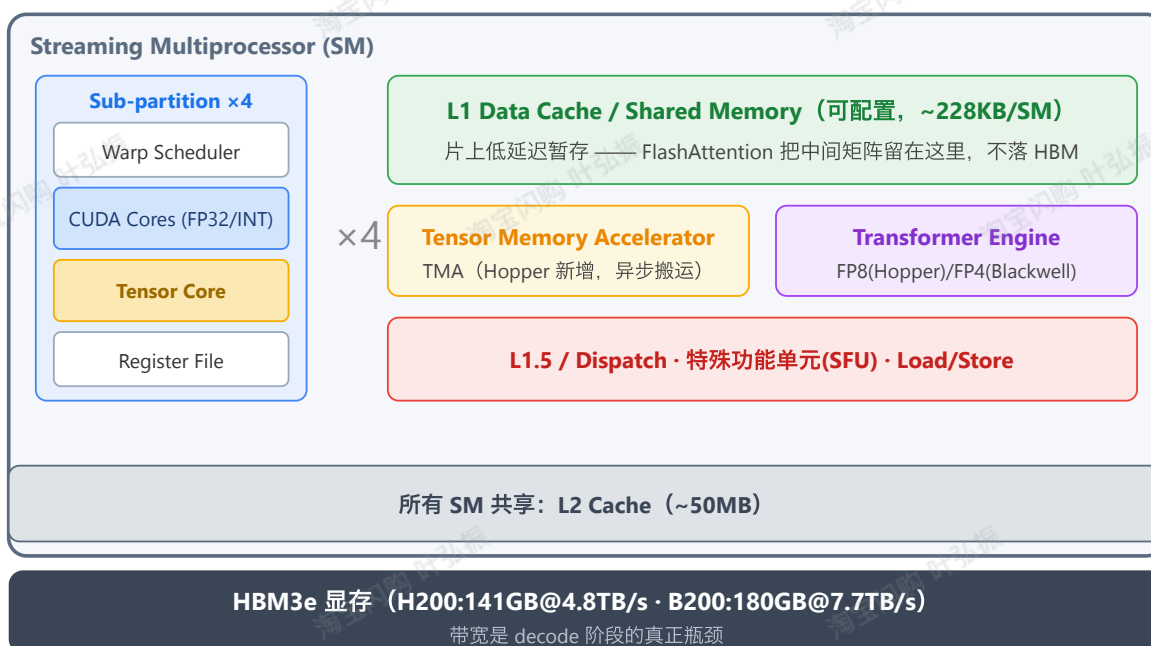
2. GPU 体系结构速成：SM、内存层级、Tensor Core

要理解上面的落差从哪来，得先看清 GPU 是怎么搭起来的。这里以 NVIDIA 的数据中心卡为线索。

2.1 SM 是 GPU 的基本计算单元

一张数据中心 GPU 由几十到上百个**流式多处理器（Streaming Multiprocessor, SM）**拼成（A100 有 108 个，H100 SXM 有 132 个）。所有 SM 共享一块大 L2 cache 和板载 HBM 显存；每个 SM 内部又是一套相对独立的小处理器。

一个 SM 的内部组织（NVIDIA Hopper 风格示意）



越靠上越快越小, 越靠下越大越慢 —— 数据在层级间搬运的代价, 决定了 memory-bound 还是 compute-bound

图2: 一个 SM 的内部组织 (Hopper 风格示意, 自绘)。规格数字引自 NVIDIA Hopper/Ampere 白皮书与 Modal GPU Glossary [A1][A3]。

一个 SM 通常切成 4 个子分区 (sub-partition), 每个子分区有自己的:

- **Warp Scheduler:** GPU 以 32 个线程为一组 (warp) 调度。调度器在多个就绪 warp 之间快速切换, 用计算掩盖访存延迟——这是 GPU 高吞吐的核心机制。
- **CUDA Cores:** 做通用 FP32/INT32 标量运算。
- **Tensor Core:** 专门做小矩阵的乘加 (如 $16 \times 16 \times 16 \times 16$)。这是算力主力。
- **Register File:** 寄存器, 最快但最稀缺的存储。

子分区之外, 整个 SM 还共享:

- **L1 Data Cache / Shared Memory:** 可配置切分的片上暂存 (Hopper 上约 228KB/SM)。低延迟、程序员可控——FlashAttention 之所以快, 关键就是把 attention 的中间结果留在这里, 不来回搬 HBM。
- **TMA (Tensor Memory Accelerator):** Hopper 新增的异步搬运引擎, 让数据搬运和计算重叠。
- **Transformer Engine:** Hopper 引入、Blackwell 增强的低精度计算单元, 动态管理 FP8 (Hopper) / FP4 (Blackwell) 的缩放。

2.2 内存层级: 越快越小, 越大越慢

性能工程的一半是在和内存层级斗争。从快到慢、从小到大:

层级	容量量级	带宽/延迟特征
寄存器	每 SM 数百 KB	最快，单周期
共享内存 / L1	~228 KB/SM	极快，~数十周期
L2 Cache	~50 MB（全卡共享）	快
HBM 显存	80–192 GB	3.35–8 TB/s，相对最慢

注意最后一行的带宽数字：H100 是 3.35 TB/s，H200 升到 4.8 TB/s，B200 到 7.7 TB/s [D1][D2]。听起来都很大，但相对于动辄上千 TFLOP/s 的算力，从 **HBM 搬数据这件事**，往往比算这件事慢得多。这个失衡是理解推理瓶颈的钥匙，§4 会量化它。

2.3 Tensor Core 演进：算力屋顶一代代抬高

Tensor Core 从 2017 年 Volta 引入以来，每一代架构都在做两件事：支持更低的精度、提供更高的吞吐。SemiAnalysis 有一篇梳理得很完整的演进史 [A2]，这里提炼主线：

架构（年份）	代表卡	Tensor Core 关键进展
Volta (2017)	V100	首次引入 Tensor Core, FP16 累加
Ampere (2020)	A100	加 TF32、BF16；结构化稀疏 (2:4) 让标称算力翻倍
Hopper (2022)	H100/H200	Transformer Engine + FP8 ；TMA 异步搬运；动态精度缩放
Blackwell (2024)	B200/GB200	第二代 Transformer Engine + FP4 (NVFP4) ；算力与显存带宽再跳一档

这条线非常重要：**每降低一档精度，算力屋顶就抬高一截**（FP8 算力约是 FP16 的 2 倍，FP4 又是 FP8 的 2 倍）。但精度不是免费午餐，§7 会讲清楚低精度的代价与边界。这里先记住一个绑定关系——**FP8 需要 Hopper 及以上，FP4 需要 Blackwell**，这直接决定了你能用哪些优化、该选哪张卡。

3. MFU 到底怎么算，分母的坑在哪

MFU 由 Google 的 PaLM 论文 (Chowdhery et al., 2022) 正式提出，现在已是衡量 GPU 训练/推理效率的事实标准 [B3][B5][B6]。它的定义看似简单：

$$\text{MFU} = \frac{\text{模型每秒实际执行的有用 FLOPs}}{\text{GPU 理论峰值 FLOP/s}}$$

但分子分母都有坑。

3.1 分子：什么算"有用的 FLOPs"

业界惯例是只算模型前向（推理）或前向+反向（训练）里算法本身要求的浮点运算，不算重计算（recomputation）、不算因实现不当而重复的运算。对 Transformer，一个广为使用的近似是：

- 推理（仅前向）每个 token 约消耗 $2N$ FLOPs，其中 N 是模型参数量（系数 2 来自一次乘加算两个浮点操作）；
- 训练（前向+反向）每 token 约 $6N$ FLOPs。

举例：一个 70B 模型做推理，每生成一个 token 约 $2 \times 70 \times 10^9 = 1.4 \times 10^{11}$ FLOPs。

如果系统每秒出 1000 个 token，分子就是 1.4×10^{14} FLOP/s = 140 TFLOP/s。

这个 $2N$ 估计忽略了 attention 里随序列长度增长的项。短上下文时够用，长上下文（尤其是 prefill 长 prompt）时 attention 的 FLOPs 不可忽略，要单独加上 $O(\text{seq}^2)$ 的项。严谨的 MFU 计算需要按实际序列长度展开 [B4][B6]。

3.2 分母：用哪个"峰值"，是 MFU 最大的争议

分母看着是个固定数，其实暗藏三个选择，选错了 MFU 会差出一倍：

1. **精度对齐**。FP16 的峰值和 FP8 的峰值差一倍，FP4 又差一倍。算 FP8 推理的 MFU，分母必须用 FP8 峰值，否则数字虚高。
2. **含不含稀疏 (sparsity)**。NVIDIA 营销材料里的算力数字经常是开了 2:4 结构化稀疏后的"标称峰值"，是稠密峰值的 2 倍。比如有些资料把 Hopper 的 FP16 写成 ~1979 TFLOPS，这其实是稀疏口径，稠密只有约 990 TFLOPS [D1]。如果你的模型没用稀疏，却拿稀疏峰值当分母，MFU 会被人为腰斩。
3. **稳态频率 vs 标称频率**。长时间满载时 GPU 会因功耗/温度降频，真实可达峰值低于 spec。

实战建议：报 MFU 时务必写清"分母 = 某精度、稠密、spec 峰值"，否则数字无法横向比较。这也是为什么不同团队报的 MFU 经常对不上——他们用了不同的分母。

3.3 推理 MFU 的现实区间

理解了口径，几个锚点数字就好读了：

- 大模型预训练：2026 年的经验是 **40–60% 算好，50%+ 算优秀**（zeroentropy 给的目标区间）[B5]。
- **推理**：普遍远低于训练。原因下一节讲——自回归 decode 天然就是 memory-bound，算力大量空转。单卡 batch=1 的 decode，MFU 个位数百分比很常见。
- 对比一个工业级反例：快手 OneRec-V2 把生成式推荐模型的**线上推理 MFU 做到 62%**，靠的是把架构彻底 dense 化（详见 §12 的呼应）——这个数字之高，恰恰说明它跳出了“推理 = 低 MFU”的常态。

4. Roofline：为什么推理有两副面孔

前面反复说“推理 MFU 低是因为 memory-bound”，这一节用 Roofline 模型把它讲透。

4.1 算术强度决定你撞哪面墙

Roofline 模型用一个量把“算得快不快”和“搬得快不快”统一起来——**算术强度 (arithmetic intensity)**：

$$I = \frac{\text{这段计算的 FLOPs}}{\text{它需要从显存搬运的字节数}} \quad (\text{单位: FLOP/Byte})$$

直觉： I 高，意味着每搬一个字节能算很多次，划算，瓶颈在算力； I 低，意味着搬得多算得少，瓶颈在带宽。两者的分界点叫**脊点 (ridge point)**：

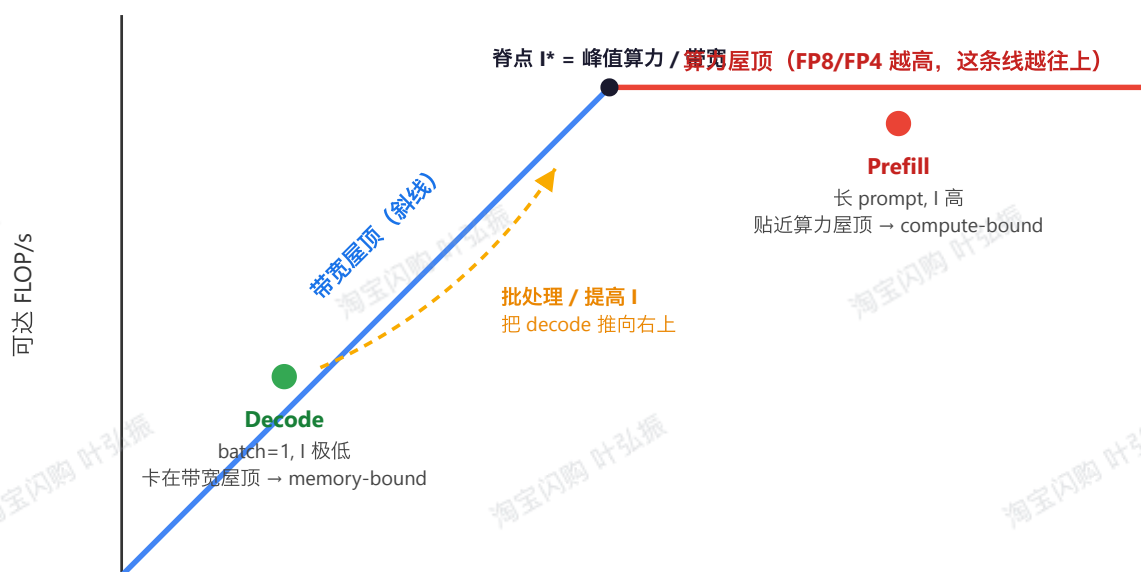
$$I^* = \frac{\text{峰值算力 (FLOP/s)}}{\text{显存带宽 (Byte/s)}}$$

以 H100 为例，FP16 稠密峰值约 990 TFLOP/s，带宽 3.35 TB/s，脊点

$I^* \approx 990,000 / 3350 \approx 295$ FLOP/Byte。也就是说：一段计算的算术强度要超过约 295，才可能吃满 H100 的 FP16 算力；达不到，就被带宽卡死。

Roofline 模型：推理的两副面孔

横轴=算术强度(FLOP/Byte)，纵轴=可达性能(FLOP/s)，双对数



算术强度 $I = \text{FLOPs} / \text{访存字节数}$ (越往右越"划算")

脊点左侧：性能由带宽决定，加算力无用；脊点右侧：性能由峰值算力决定，加带宽无用

图3：Roofline 模型与推理的两副面孔（自绘）。脊点左侧由带宽决定性能，右侧由峰值算力决定性能。

4.2 Prefill 是 compute, Decode 是 bandwidth

LLM 推理分两个阶段，它们落在 Roofline 的两侧——这是整个推理优化领域最重要的一句话，被反复提炼为 "*Prefill is Compute, Decode is Bandwidth*" [C3]：

Prefill（处理输入 prompt）：一次性把整个 prompt 的所有 token 喂进去，矩阵乘法是"高瘦×宽"的大 GEMM，算术强度高，落在脊点右侧——**compute-bound**，能把 Tensor Core 喂得比较满，MFU 相对高。

Decode（自回归生成）：一次只生成一个 token。batch=1 时，每一层的权重矩阵要从 HBM 完整搬一遍，却只拿来和一个 token 的向量相乘——算了一丁点，搬了一大堆。算术强度极低（接近 1 FLOP/Byte 量级），死死卡在带宽屋顶左侧——**memory-bound**。

量化一下 decode 的窘境：生成一个 token 要把整个模型的权重读一遍。70B 模型 FP16 权重约 140 GB，H100 带宽 3.35 TB/s，光搬权重就要 $140/3350 \approx 42$ ms——这意味着单卡 batch=1 的理论上限只有约 24 token/s，且几乎全部时间在搬权重，Tensor Core 基本闲着。这就是 decode MFU 个位数的根因 [C1][C2][C6]。

4.3 钥匙：把 decode 推向右上

既然 decode 的病根是算术强度太低，所有 decode 优化的母题就一句话：**提高算术强度，把工作点从带宽屋顶推向脊点。**

最直接的办法是**增大 batch**：搬一次权重，同时给 B 个请求的 token 用，算术强度近似放大 B 倍。这就是为什么推理服务拼命做大并发、做 continuous batching (§5)。但 batch 不能无限大——KV Cache 显存、延迟 SLA 都是约束。第二部分讲的全部技术，几乎都可以归到“如何在各种约束下提高有效算术强度 / 减少无效搬运”这一条主线上。

第二部分：常见提升办法

第一部分已经把问题归结成一句话：**推理 MFU 低，主要是 decode 阶段算术强度太低、HBM 搬运占满了时间。**这一部分的每一个技术，都是在攻这条主线上的某一段——要么提高算术强度 (§5)，要么压缩搬运量或消灭无效搬运 (§5、§6)，要么直接抬高算力屋顶 (§7)，要么在多卡上把账算清楚 (§8)。

5. 提高算术强度：把 memory-bound 推回 compute-bound

5.1 批处理：从静态到连续

最朴素的提算术强度手段是**批处理**——多个请求共享一次权重搬运。但 LLM 请求长度参差不齐、到达时间随机，传统的**静态批处理 (static batching)**有个致命浪费：必须等一个 batch 里最长的请求生成完，整批才能返回，短请求生成完只能干等，那些 SM 空转。

连续批处理 (continuous batching)，又称 in-flight batching 解决了这个问题：调度器以 iteration 为粒度工作，每生成一步就检查——有请求结束了就让它立刻退出、把腾出的位置补进新到的请求。GPU 始终保持高并发，不为长尾请求陪绑 [E3][E5]。这是 vLLM、TensorRT-LLM 等现代推理引擎的标配，相比静态批处理常有数倍吞吐提升。

5.2 KV Cache 与 PagedAttention：让大 batch 真的塞得下

提高 batch 的最大物理约束是显存——每个请求都要存自己的 **KV Cache**（已生成 token 的 key/value，避免重复计算）。KV Cache 随上下文线性增长，一个长上下文请求能吃掉几个 GB。传统实现给每个请求预分配一段连续显存，按最大长度留空间，导致严重的内部碎片（实测有效利用率常不到 40%）。

PagedAttention（vLLM 的核心创新）借了操作系统虚拟内存分页的思路：把 KV Cache 切成固定大小的 block，按需分配、非连续存储，用一张 block table 做地址映射 [E3][E4]。碎片几乎消失，同样的显存能容纳的并发请求数大幅提升——而并发上去了，算术强度也跟着上去。省显存省出来的，最终变成了 MFU，这条间接路径很容易被漏看。

5.3 FlashAttention: 把 attention 从 memory-bound 里救出来

标准 attention 的实现会显式地算出并存储 $N \times N$ 的注意力矩阵（ N 是序列长度），来回读写 HBM。这个 $N \times N$ 矩阵的搬运量随序列长度平方增长，是个不折不扣的 memory-bound 重灾区。

FlashAttention (Dao et al., NeurIPS 2022) 的核心是 **IO-aware + tiling**: 把 Q、K、V 切块，在 SM 的共享内存里分块计算 attention，用在线 softmax (online softmax) 增量地累积结果，**全程不把那个 $N \times N$ 大矩阵落回 HBM** [E2][E4]。它算的 FLOPs 一点没少（是 exact attention，不是近似），但把主要的 HBM 读写量从 $O(N^2)$ 降到 $O(N)$ 量级，算术强度大幅提高。后续 FlashAttention-2/3 进一步优化了在 Hopper 上的 warp 划分和异步流水。过去几年里，单个 kernel 级优化能拿到这么大收益的，没几个。

6. 算子与执行层优化: 缩小 SM Active $\rightarrow$ Tensor Active 的落差

6.1 Kernel 融合与 CUDA Graphs: 消灭启动间隙

一次 GPU kernel 启动有固定开销（微秒级）。Transformer 一层里有大量小算子（bias 加法、激活、LayerNorm、缩放……），如果每个都单独启动一个 kernel，启动开销累加起来能占掉可观的时间，且 kernel 之间 GPU 可能空转——这正是 GPU-Util 高而 SM Active 没那么高的一大来源。

两个互补的解法：

- **Kernel 融合 (fusion)**：把多个连续的小算子合并成一个 kernel，中间结果留在寄存器/共享内存，既省启动开销又省 HBM 往返。FlashAttention 本身就是一种深度融合。torch.compile、TensorRT 的图优化会自动做大量融合。
- **CUDA Graphs**：把一连串 kernel 的启动序列录制成一张图，之后整张图一次性提交，绕过逐个启动的 CPU 开销。对 decode 这种“同样的小图反复跑几百次”的场景收益尤其明显。

6.2 GEMM 形状: 为什么“奇异维度”打不满 Tensor Core

Tensor Active $\rightarrow$ MFU 那道落差里，最隐蔽的就数这一项。Tensor Core 以固定大小的 tile 工作（比如一次算 128×256 的输出块）。如果你的矩阵维度不是 tile 的整数倍，硬件会**向上取整**到整 tile，多算的部分被丢弃——这叫 **tile quantization**；类似地，当输出 tile 数量不能均匀分配到所有 SM 上时，最后一“波” (wave) 只有部分 SM 在干活，叫 **wave quantization**。

后果是：把隐藏维从 5120 改成 5152，或 batch 从 128 改成 130，都可能让一个 GEMM 的有效 MFU 显著掉一截，因为它跨过了 tile/wave 的整数边界。**实战启示：**模型的隐藏维、FFN 维、batch size 尽量对齐到 128 或更大的 2 的幂，是几乎零成本的 MFU 优化。CUTLASS 等库的存在，很大程度就是为了在各种形状下榨出最优的 tile 策略。

7. 低精度：直接抬高算力屋顶

前面的优化都在"把工作重点往屋顶推"，低精度是另一条路——**直接把屋顶抬高**。精度每降一档，单位时间能塞进 Tensor Core 的运算量翻倍，同时权重和 KV Cache 的搬运字节数减半（等于算术强度也涨）。算力和带宽两头都赚的优化很少，这算一个。

7.1 FP16 → FP8 → FP4 的阶梯

精度	需要的架构	相对 FP16 算力	典型用途
FP16/BF16	通用	1×	训练、对精度敏感的推理
FP8 (E4M3/E5M2)	Hopper+	~2×	主流推理，精度损失小
FP4 (NVFP4/MXFP4)	Blackwell	~4×	前沿推理，需精心校准

精度与架构是死绑定的：FP8 的 Transformer Engine 从 Hopper 才有，FP4 要等 Blackwell。这意味着**"能不能用 FP4 加速"本质是个选卡问题**（§10）。

7.2 NVFP4：4-bit 怎么做到不崩

直接把权重压到 4-bit，数值动态范围会崩——这是 FP4 落地的核心难题。NVIDIA 的 **NVFP4** 用一套两级缩放来救 [H1][H2]：

- 数值本身是 **E2M1**（1 符号 + 2 指数 + 1 尾数）的 4-bit 浮点；
- 每 **16 个值** 组成一个微块（micro-block），共享一个 **FP8 (E4M3)** 的缩放因子；
- 再叠加一个**每张量的 FP32 二级缩放因子**。

关键设计是那个 16 值的微块——比竞品 MXFP4 的 32 值块更细，能更局部地贴合数据的动态范围，量化误差更小 [H1][H3]。效果上，NVIDIA 在 DeepSeek-R1 上的测试显示，从 FP8 转到 NVFP4，**MMLU-PRO 仅从 85% 掉到 84%、AIME 2024 反而 89%→91%、Math-500 持平 98%**，号称"关键语言建模任务上精度损失 ≤1%" [H1]。内存上，NVFP4 相对

FP16 减少约 3.5×、相对 FP8 减少约 1.8×；能效上，Blackwell 的 FP4 相对 H100 baseline 有 25× 量级的提升 [H1]。

7.3 低精度不是免费的

代价要说清楚：

- **逐层敏感度不均**。一篇 FP4 诊断研究 (arXiv 2603.08747) 做了逐层、逐块的敏感度分析，发现不是所有层都能安全降到 FP4——attention 的某些投影、首尾层往往更敏感 [H3]。务实的做法是**混合精度**：敏感层保 FP8/FP16，其余上 FP4。
- **需要校准甚至 QAD**。直接 PTQ (训练后量化) 到 FP4 常有精度损失，**量化感知蒸馏 (QAD) **能把损失补回来，代价是额外训练 [H2]。

这与我们在快手 OneRec-V2 量化篇里看到的结论一致：低精度推理能落地的前提，是模型数值分布足够"温和"且经过适配，不是无脑切精度就行。

8. 并行与放置：把利用率账本算到多卡

单卡之外，大模型必须切到多卡，并行方式直接影响整体 MFU：

- **张量并行 (TP)**：把每个矩阵乘切到多卡，每步都要 all-reduce 通信。通信量大但延迟低，适合对延迟敏感的 **decode** 和单机多卡 (NVLink 域内)。TP 度数越高，通信占比越大，MFU 会被通信吃掉一部分。
- **流水并行 (PP)**：按层切分，卡间只传激活，通信量小，但有 pipeline bubble (首尾阶段空转)。适合跨节点。
- **专家并行 (EP)**：MoE 模型特有，把不同专家放到不同卡。负载均衡是关键——专家热度不均会让部分卡闲置，拖垮整体利用率。
- **数据并行 (DP)**：多副本各跑各的 batch，推理服务里常用于横向扩并发。

一个常被忽略的点：这些并行的通信开销，会体现在 GPU-Util → SM Active 那道落差里——通信等待时 SM 在空转。所以多卡场景下，光看单卡 MFU 不够，要把通信开销纳入"系统级有效利用率"的账本。这也为第三部分的分离式推理埋下伏笔：既然 prefill 和 decode 对并行方式的偏好不同 (prefill 偏 TP、decode 偏 EP/DP)，为什么不干脆把它们拆开部署？

第三部分：前沿与实战

9. 分离式推理：当前最大的结构性变革

§8 末尾留了个问题：prefill 偏 compute-bound、偏 TP，decode 偏 memory-bound、偏 EP/DP——两者性格相反，却被塞在同一张卡上轮流跑。这会带来两个具体的痛：一是互相拖累，一个长 prefill 进来会卡住正在进行的 decode，让 decode 的逐 token 延迟（TBT, time-between-tokens）剧烈抖动；二是工作点错配，同一张卡没法同时是"算力最优"和"带宽最优"的配置。

****分离式推理（Disaggregated Serving，又称 PD 分离）****的思路很直接：把 prefill 和 decode 拆到不同的卡/节点上，各自用最适合自己的硬件配置和并行策略，prefill 算完把 KV Cache 传给 decode 节点接着生成。

PD 分离：让每个阶段跑在自己 MFU 最高的工作点

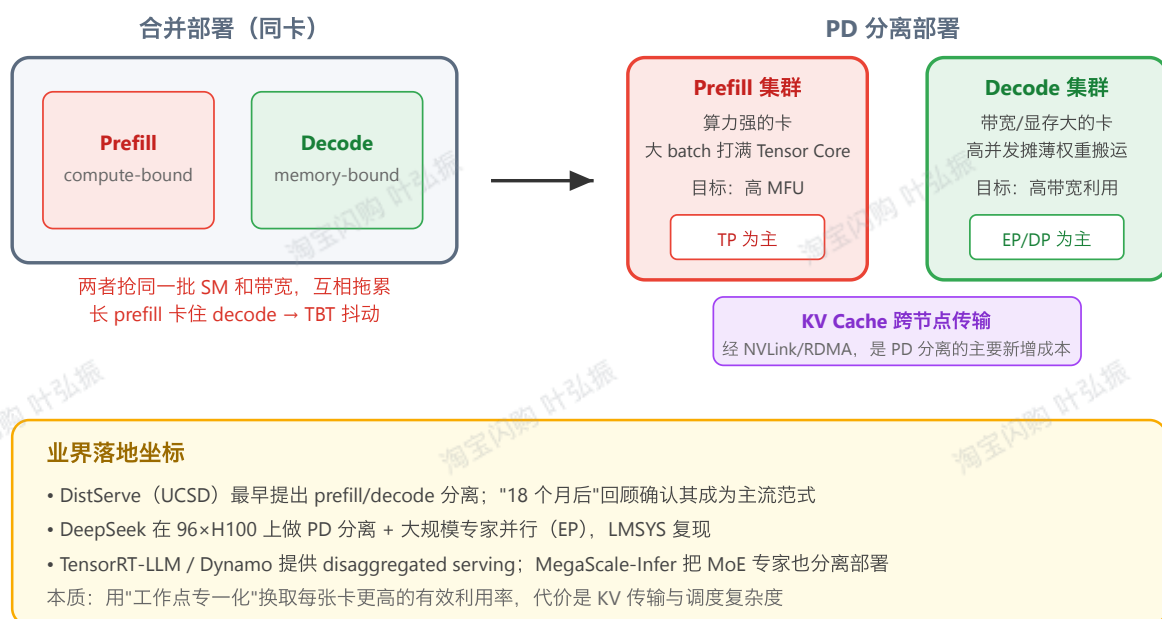


图 4：PD 分离的动机与业界落地（自绘）。本质是让每个阶段跑在自己 MFU/带宽利用率最高的工作点上，代价是 KV Cache 跨节点传输与调度复杂度。

9.1 它为什么能提升整体利用率

把账算清楚就明白了。合并部署时，为了压住 decode 的 TBT 抖动，你往往不敢把 prefill 的 batch 开太大，于是 prefill 的 MFU 上不去；反过来为了喂饱 prefill，decode 的并发又受限。两个目标在一张卡上是冲突的。

分离之后：

- **Prefill 集群**可以放算力强的卡、开大 batch、用 TP，把 Tensor Core 喂满，追求高 MFU；
- **Decode 集群**可以放显存大/带宽高的卡、堆高并发、用 EP/DP，把权重搬运摊薄到尽可能多的请求上，追求高带宽利用率；

- 两个集群各自独立扩缩容，按 prefill/decode 的实际负载比例配比。

DistServe (UCSD Hao AI Lab) 是最早系统论证这件事的工作之一；它的"18 个月后"回顾文章确认了 PD 分离已经从一个研究想法变成主流生产范式 [F3]。

9.2 业界落地坐标

- **DeepSeek 大规模实践**：在 96 张 H100 上做 PD 分离 + 大规模专家并行 (large-scale EP)，LMSYS 做了公开复现和详细拆解 [F1]。公开资料里，这大概是规模最大、拆得最细的落地案例。
- **TensorRT-LLM / NVIDIA Dynamo**：官方提供 disaggregated serving 能力，并发文讨论"如何从猜测式调参变成有方法地配比 prefill/decode 节点" [F2][F5]。
- **MegaScale-Infer** (arXiv 2504.02263)：把分离思想推进到 MoE——不光 PD 分离，连专家本身也分离部署，用 disaggregated expert parallelism 服务超大 MoE [F4]。
- 也有冷静的声音：arXiv 2506.05508 提醒分离不是银弹，KV 传输开销、调度复杂度、小规模下的收益不明显都是真实成本，要按负载规模权衡 [F6]。

一句话总结：分离式推理的本质，是用"工作点专一化"换取每张卡更高的有效利用率——这正是 MFU 优化从单卡算子层上升到集群编排层的标志。

10. NVIDIA 卡型横评：按推理工作负载选卡

理解了 prefill/decode 的不同需求，选卡就有了清晰的判据。下表汇总主力数据中心卡的关键规格（FP16 列标注稠密口径，避免 §3.2 说的稀疏混淆）：

规格	A100 (Ampere)	H100 SXM (Hopper)	H200 SXM (Hopper)	B200 (Blackwell)	GB200 (每 B200 die)
HBM 容量	80 GB HBM2e	80 GB HBM3	141 GB HBM3e	180 GB HBM3e	~186 GB HBM3e
显存带宽	~2.0 TB/s	3.35 TB/s	4.8 TB/s	7.7 TB/s	~8.0 TB/s
FP16 稠密算力	~312 TFLOPS	~990 TFLOPS	~990 TFLOPS	~2250 TFLOPS	~2500 TFLOPS
FP8 算力	不支持	✓ ~2× FP16	✓ ~3958 TFLOPS	✓ ~4500 TFLOPS	✓ ~5000 TFLOPS

FP4 (NVFP4)	X	X	X	✓ 9000 TFLOPS	✓ ~10000 TFLOPS
NVLink	600 GB/s	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s + C2C
TDP	400 W	700 W	700 W	1000 W	1200W+300 W(CPU)

规格来源：spheron / GMI Cloud / exxact 卡型对比 [D1][D2][D5]，FP16 稠密值按 §3.2 口径校正。GB200 是含 2×B200 + Grace CPU 的 Superchip，非独立 GPU。

10.1 三条选卡判据

判据一：模型放得下吗（看显存）。 decode 受显存约束最直接——模型权重 + KV Cache 要全塞进显存。同样跑一个大 MoE，H200 的 141GB 比 H100 的 80GB 能少切几张卡、少付通信开销。这是 H200 相对 H100 算力没变、却在推理上很受欢迎的原因：它升级的是带宽和容量，恰好是 decode 的两个命门 [D2][D5]。

判据二：decode 还是 prefill 为主（看带宽 vs 算力）。 decode 重的负载（长输出、聊天）优先看带宽——H200→B200 的带宽从 4.8 到 7.7 TB/s 是实打实的 decode 提速。prefill 重的负载（长文档、RAG、批量处理）优先看算力和低精度——B200 的 FP4 屋顶是 H 系完全没有的维度。

判据三：能不能用上低精度（看架构代际）。 三条判据里，这一条最容易被忽略，收益却最大。如果你的模型能安全跑 FP4 (§7.3)，那 B200 相对 H200 的有效算力提升远不止 spec 上那点差距——因为 FP4 这条屋顶 Hopper 根本没有。换句话说，"要不要上 Blackwell"很大程度等价于"我的模型能不能吃下 FP4"。

10.2 一个常见误区

不要只看 FP16/FP8 的 TFLOPS spec 对比就下单。对 decode 为主的在线服务，带宽和显存容量往往比峰值算力更决定实际吞吐——因为你大概率卡在 Roofline 的带宽屋顶那一侧，算力屋顶再高也用不上。先用 §4 的方法判断自己的负载落在 Roofline 哪一侧，再决定为哪个维度付钱。

11. Profiling 实操：别再被 GPU-Util 骗

讲了一堆原理和方案，落到日常，第一步永远是：**先量准，再优化**。而量准的前提是用对工具、看对指标。

11.1 三件套各看什么

工具	粒度	主要回答
nvidia-smi	卡级、秒级	卡活着没、显存占用、功耗 ——不要用它判断算力效率
DCGM (<code>dcgm</code> <code>dmon</code>)	卡级、可连续采集	<code>SM_ACTIVE</code> 、 <code>SM_OCCUPANCY</code> 、 <code>PIPE_TENSOR_ACTIVE</code> 、 <code>DRAM_ACTIVE</code> ——生产监控的正确指标层
Nsight Systems	时间线、跨 kernel	找 kernel 间隙、CPU-GPU 同步、通信与计算重叠情况
Nsight Compute	单 kernel、逐指令	某个 kernel 到底卡在算力还是访存、occupancy 受什么限制

诊断的正确顺序是从粗到细：DCGM 连续监控发现 `SM_ACTIVE` 高但 `PIPE_TENSOR_ACTIVE` 低 → 用 Nsight Systems 看时间线定位是哪类 kernel 占了时间 → 用 Nsight Compute 钻进那个 kernel 看它是 memory-bound 还是被 occupancy 限制 [G2][G4]。

11.2 关键指标与判读

回到第一部分的四层漏斗，对应到具体指标：

- `DRAM_ACTIVE` (HBM 读写占比) 高 + `PIPE_TENSOR_ACTIVE` 低 → 典型 **memory-bound**，往 §5 (提算术强度、FlashAttention、batching) 方向治；
- `SM_ACTIVE` 高 + `PIPE_TENSOR_ACTIVE` 低、且 `DRAM` 也不高 → 大概率在跑非 **GEMM** 算子或被启动间隙拖累，往 §6 (融合、CUDA Graphs) 方向治；
- `SM_OCCUPANCY` 低 → 寄存器/共享内存用量限制了驻留 warp 数，调 kernel 配置；
- 各项都不低但 MFU 还是上不去 → 查 §6.2 的 **GEMM** 形状，多半是 tile/wave quantization。

学术界也在推动“把利用率测得更深一层”——arXiv 2501.16909 *Measuring GPU Utilization One Level Deeper* 系统论证了为什么 nvidia-smi 的 util 不可靠、应该下钻到子单元级指标 [G1]。这篇可以作为建立团队 profiling 规范的理论依据。

11.3 给推理服务的最小监控集

如果只能上几个指标做线上常态监控，建议这一组（都来自 DCGM，开销低）：`SM_ACTIVE`、`PIPE_TENSOR_ACTIVE`、`DRAM_ACTIVE`、显存占用、功耗。配合业务侧的 token/s 和 TBT，基本能在"该扩容还是该优化"这个高频决策上不被 GPU-Util 误导。

12. 结语：MFU 的天花板与下一站

把全文收束成几条：

一、"利用率"必须分层看。GPU-Util $\geq$ SM Active $\geq$ Tensor Active $\geq$ MFU，真正有诊断价值的是相邻两层的落差，而不是任何单一数字。被 nvidia-smi 的 99% 骗去扩容，是最常见也最贵的错误。

二、推理 MFU 低的根在 Roofline。prefill compute-bound、decode memory-bound 是结构性的。decode 单卡 batch=1 几乎全时间在搬权重，MFU 个位数是物理决定的，不是实现 bug。

三、优化是一条主线上的多个着力点。提算术强度（batching、PagedAttention、FlashAttention）、消无效搬运与间隙（融合、CUDA Graphs）、抬算力屋顶（FP8/FP4）、算清多卡账本（并行 + 分离式）——每一招都对应漏斗里的某一道落差，可以按 profiling 结果对症下药。

四、趋势是从"单卡算子"上升到"集群工作点编排"。分离式推理是这个上升的标志：当单卡优化逼近天花板，下一个数量级的收益来自让 prefill/decode/专家各自跑在最优工作点上。低精度（FP4 及更低）则是另一条仍在快速推进的轴，但越往下越依赖逐层敏感度分析和量化感知训练。

五、对我们自己的呼应。这套框架直接服务于推荐模型的推理优化。快手 OneRec 系列把生成式推荐模型的线上推理 MFU 做到 62% (§3.3)，路径正是本文的主线：Lazy Decoder 把计算 dense 化（提算术强度、让 decode 不再 memory-bound）+ FP8 量化（抬算力屋顶）。这反过来印证——推理 MFU 不是天生就低，而是看你愿不愿意为"让 Tensor Core 真正忙起来"重构架构。对延迟敏感的闪购排序场景，先用 §4 判断瓶颈落在 Roofline 哪侧、再用 §11 的指标量准，是任何推理优化动手前都该走的两步。

至于 MFU 的天花板：训练已经能稳定做到 50%+，推理在架构配合下（dense 化 + 大并发 + 低精度）也能摸到同一区间。真正的上界不在硬件，而在你的模型结构离"理想的 compute-bound 形状"有多远。

参考文献

A. GPU 体系结构 / SM

- [A1] Modal — GPU Glossary: Streaming Multiprocessor. <https://modal.com/gpu-glossary/device-hardware/streaming-multiprocessor>
- [A2] SemiAnalysis — NVIDIA Tensor Core Evolution: From Volta To Blackwell. <https://newsletter.semianalysis.com/p/nvidia-tensor-core-evolution-from-volta-to-blackwell>
- [A3] NVIDIA — Ampere Architecture / A100 Tensor Core GPU Whitepaper. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf>

B. MFU 定义与测量

- [B1] Modal — GPU Utilization Guide. <https://modal.com/blog/gpu-utilization-guide>
- [B2] Databricks — LLM Inference Performance Engineering: Best Practices. <https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices>
- [B3] Jaideep Ray (Better ML) — Using Model FLOPs Utilization (MFU). <https://medium.com/better-ml/using-model-flops-utilization-mfu-7b17de07faec>
- [B4] Performance Modeling of Distributed LLM Training and Inference. arXiv:2407.14645. <https://arxiv.org/html/2407.14645v1>
- [B5] zeroentropy — MFU: model FLOPs utilization as the GPU efficiency metric. <https://zeroentropy.dev/concepts/mfu/>
- [B6] Glenn K. Lockwood — Model FLOPs Utilization. <https://www.glennklockwood.com/garden/mfu>

C. 推理瓶颈 / Roofline

- [C1] APXML — Memory Bandwidth and Compute Bottlenecks in LLM Inference. <https://apxml.com/courses/llm-compression-acceleration/chapter-1-foundations-llm-efficiency-challenges/memory-compute-bottlenecks-inference>
- [C2] A Systematic Characterization of LLM Inference on GPUs. arXiv:2512.01644. <https://arxiv.org/html/2512.01644v1>
- [C3] Asheesh Goja — Prefill is Compute, Decode is Bandwidth. <https://www.linkedin.com/pulse/prefill-compute-decode-bandwidth-architectural-case-llm-asheesh-goja-gpo1c>
- [C4] Predicting LLM Inference Latency: A Roofline-Driven ML Method. NeurIPS 2024 ML for Systems. <https://mlforsystems.org/assets/papers/neurips2024/paper28.pdf>
- [C5] LLM Performance Model — Prefill & Decode Estimator. <https://joursbleu.github.io/llm-perf-model/>

- [C6] Why LLM Inference Is Memory-Bound (Not Compute-Bound).
<https://medium.com/@arjunravi726/why-llm-inference-is-memory-bound-not-compute-bound-ba59c48739e0>

D. NVIDIA 卡型

- [D1] Exxact — Comparing NVIDIA Tensor Core GPUs (Blackwell vs Hopper).
<https://www.exxactcorp.com/blog/hpc/comparing-nvidia-tensor-core-gpus>
- [D2] Spheron — H200 vs B200 vs GB200: Memory, Bandwidth & Cost (2026).
<https://www.spheron.network/blog/nvidia-h200-vs-b200-vs-gb200/>
- [D3] HPC-AI Tech — B200 Practical Considerations for AI Teams. <https://www.hpc-ai.com/blog/B200-Practical-Considerations-for-AI-Teams>
- [D4] NVIDIA Developer — Inference Performance for Data Center Deep Learning.
<https://developer.nvidia.com/deep-learning-performance-training-inference/ai-inference>
- [D5] GMI Cloud — H100 vs H200 vs B200 for LLM Inference.
<https://www.gmicloud.ai/en/blog/h100-vs-h200-vs-b200-llm-inference-workload>

E. 算子 / attention / batching

- [E1] NVIDIA — Mastering LLM Techniques: Inference Optimization.
<https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>
- [E2] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.
https://papers.nips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf
- [E3] Insu Jang — LLM Inference: Continuous Batching and PagedAttention.
<https://insujang.github.io/2024-01-07/llm-inference-continuous-batching-and-pagedattention/>
- [E4] HuggingFace — Paged Attention docs; Dao-AILab/flash-attention.
https://huggingface.co/docs/transformers/paged_attention
- [E5] Towards AI — LLM Inference Optimization: KV Cache, Batching.
<https://pub.towardsai.net/llm-inference-optimization-730b2d718a44>

F. 分离式推理 / MoE

- [F1] LMSYS — Deploying DeepSeek with PD Disaggregation and Large-Scale EP on 96 H100 GPUs. <https://lmsys.org/blog/2025-05-05-large-scale-ep/>

- [F2] NVIDIA — Disaggregated Serving in TensorRT-LLM.
https://nvidia.github.io/TensorRT-LLM/blogs/tech_blog/blog5_Disaggregated_Serving_in_TensorRT-LLM.html
- [F3] Hao AI Lab (UCSD) — Disaggregated Inference: 18 Months Later.
<https://haoailab.com/blogs/distserve-retro/>
- [F4] MegaScale-Infer: Serving MoE at Scale with Disaggregated Expert Parallelism.
arXiv:2504.02263. <https://arxiv.org/pdf/2504.02263>
- [F5] NVIDIA — Removing the Guesswork from Disaggregated Serving.
<https://developer.nvidia.com/blog/removing-the-guesswork-from-disaggregated-serving/>
- [F6] Beyond the Buzz: A Pragmatic Take on Inference Disaggregation.
arXiv:2506.05508. <https://www.arxiv.org/html/2506.05508>

G. Profiling

- [G1] Measuring GPU Utilization One Level Deeper. arXiv:2501.16909.
<https://arxiv.org/html/2501.16909v1>
- [G2] NVIDIA — Nsight Compute Kernel Profiling Guide. <https://docs.nvidia.com/nsight-compute/>
- [G3] Packet.ai — GPU Utilization: The Lie Your Dashboard Tells You.
<https://packet.ai/blog/gpu-utilization-the-lie-your-dashboard-tells>
- [G4] NVIDIA — Measuring the GPU Occupancy of Multi-stream Workloads.
<https://developer.nvidia.com/blog/measuring-the-gpu-occupancy-of-multi-stream-workloads/>
- [G5] TechnoLynx — AI GPU Utilization Testing: What GPU-Util Means and What It Misses. <https://www.technolynx.com/post/gpu-utilization-ai-benchmark-testing>

H. 低精度 / FP4

- [H1] NVIDIA — Introducing NVFP4 for Efficient and Accurate Low-Precision Inference.
<https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>
- [H2] Red Hat — Accelerating LLMs with NVFP4 Quantization.
<https://developers.redhat.com/articles/2026/02/04/accelerating-large-language-models-nvfp4-quantization>
- [H3] Diagnosing FP4 Inference: Layer-Wise and Block-Wise Sensitivity of NVFP4 and MXFP4. arXiv:2603.08747. <https://arxiv.org/html/2603.08747v1>

检索经由 easy_anysearch 完成 (2026-06-15) ; 硬件 spec 经 WebFetch 二次核对。图为自绘 SVG, 数据出处见各图注与上表来源标注。公开发表前请按各来源 license 复核引用。